# BigQuery - Cheat Sheet

BigQuery is Google Cloud's serverless data warehouse. You write SQL, BigQuery picks how many nodes to run it on, and you pay for the bytes scanned (or for reserved slots if you have predictable volume). Storage and compute are decoupled, so a 10 PB table costs the same to keep around whether you query it daily or once a year.

The mental model is different from Postgres. Think of it as a query engine over columnar files in object storage, with SQL as the user interface.

---

## Architecture in one paragraph

BigQuery sits on top of Colossus (storage), Dremel (query engine), Borg (scheduling), and Jupiter (networking). Tables live as columnar files in Colossus. Queries run on "slots", which are virtual CPU + memory units the engine schedules dynamically. There are no machines you provision. There are no indexes in the traditional sense.

---

## Datasets, tables, views

The hierarchy:

```
project
  dataset (like a database or schema)
    table
    view
    materialized view
    external table
    routine (function, stored procedure)
```

Fully qualified name: `project.dataset.table`. Backticks for names with hyphens: `` `my-project.analytics.events` ``.

```
CREATE TABLE `my-project.analytics.users` (
    id INT64,
    email STRING,
    created_at TIMESTAMP,
    tags ARRAY<STRING>,
    profile STRUCT<name STRING, age INT64>
);

INSERT INTO `my-project.analytics.users` (id, email, created_at, tags, profile)
VALUES (1, 'alice@example.com', CURRENT_TIMESTAMP(), ['premium', 'us'], ('Alice',
        30));
```

Region matters. A dataset has a location (US, EU, asia-east1, etc.) and you can only join tables in the same location.

---

## GoogleSQL (the SQL flavor)

BigQuery's SQL dialect is now called GoogleSQL (formerly "Standard SQL"). It's ANSI-SQL-compliant with Google-specific extensions. There's also a legacy SQL dialect from the early days. Always use GoogleSQL for new work.

### Data types worth knowing

| Type | Notes |
|---|---|
| `INT64` | 64-bit signed integer (no smaller ints) |
| `NUMERIC` / `BIGNUMERIC` | Exact decimal |
| `FLOAT64` | Double-precision float |
| `STRING` | UTF-8 |
| `BYTES` | Raw bytes |
| `BOOL` | true / false |
| `DATE`, `TIME`, `DATETIME`, `TIMESTAMP` | Time types |
| `ARRAY<T>` | Ordered list of T |
| `STRUCT<...>` | Named-field record |
| `JSON` | First-class JSON column |
| `GEOGRAPHY` | Geospatial |

No `VARCHAR`. No `SMALLINT`. The query engine doesn't care about width.

### ARRAY, STRUCT, UNNEST

These trip up devs coming from Postgres or MySQL. BigQuery embraces nested and repeated columns.

```sql
-- Array literal
SELECT [1, 2, 3] AS nums;

-- Struct literal
SELECT STRUCT('alice' AS name, 30 AS age) AS person;

-- Array of structs (common shape for event data)
SELECT [
    STRUCT('login' AS type, TIMESTAMP '2025-01-01 09:00:00' AS at),
    STRUCT('view'  AS type, TIMESTAMP '2025-01-01 09:01:00' AS at)
] AS events;
```

`UNNEST` flattens arrays into rows:

```sql
SELECT user_id, tag
FROM users, UNNEST(tags) AS tag
WHERE tag = 'premium';
```

Cross-join with an array column is the standard pattern. Each input row produces one output row per array element.

---

## Partitioning

Partitions split a table by a column value so queries only scan the relevant chunks. This is the single biggest cost lever.

### Time-based partitioning (most common)

```sql
CREATE TABLE `my-project.analytics.events` (
    event_time TIMESTAMP,
    user_id INT64,
    payload JSON
)
PARTITION BY DATE(event_time);
```

Query a single day, scan one partition:

```sql
SELECT COUNT(*)
FROM `my-project.analytics.events`
WHERE DATE(event_time) = '2025-01-01';
```

Without the `WHERE`, you'd scan the entire table. With it, you scan only that day. Cost ratio: 1 to 365 or worse.

### Other partition types

- **Ingestion time**: `PARTITION BY _PARTITIONDATE` (no user column needed).
- **Integer range**: `PARTITION BY RANGE_BUCKET(user_id, GENERATE_ARRAY(0, 100000, 1000))`.
- **Daily / hourly / monthly / yearly**: by `DATE_TRUNC`.

### Require partition filter

For production tables, force callers to include the partition column:

```
CREATE TABLE ... PARTITION BY DATE(event_time)
OPTIONS (require_partition_filter = true);
```

A query without `WHERE DATE(event_time) = ...` fails fast instead of scanning the full table and racking up a five-figure bill.

---

## Clustering

Clustering sorts data within a partition by one or more columns. Filters on clustered columns scan only matching blocks.

```
CREATE TABLE `my-project.analytics.events` (...)
PARTITION BY DATE(event_time)
CLUSTER BY user_id, event_type;
```

- Up to 4 cluster columns.
- Order matters. Filter prefix-wise.
- Cluster on high-cardinality columns used in filters and joins (user_id, account_id, event_type).

Partitioning + clustering is the standard combo for high-volume event tables.

---

## Pricing model

Two main modes.

### On-demand

Pay per TB of data scanned by queries. Storage is billed separately at a flat rate per GB-month.

- Active storage: ~$0.02 / GB-month (varies by region).
- Long-term storage (untouched for 90 days): ~$0.01 / GB-month.
- Query: ~$6.25 / TB scanned (US, 2025).

The “scanned” part is the trap. `SELECT *` over a 10 TB table costs around $62.50 per query. `SELECT col1` from the same table touches only that column’s bytes, which might be 100 GB.

### Capacity (slots)

Reserve a fixed number of slots for a flat monthly fee. Predictable cost, no per-query bill. Worth it once on-demand bills hit thousands per month.

Slot reservations come in editions (Standard, Enterprise, Enterprise Plus) with different feature sets and per-slot prices.

### Free tier

1 TB of queries and 10 GB of storage per month, free. Enough for small projects and exploration.

---

## Loading data

### Batch load (cheap, default)

Load CSV, JSON, Avro, Parquet, or ORC from Cloud Storage. Free. No query cost.

```
bq load \
  --source_format=PARQUET \
  my-dataset.events \
  gs://my-bucket/events/2025-01-01/*.parquet
```

Or from SQL:

```
LOAD DATA INTO `my-project.analytics.events`
FROM FILES (
    format = 'PARQUET',
    uris = ['gs://my-bucket/events/2025-01-01/*.parquet']
);
```

### Streaming insert (legacy)

`tabledata.insertAll` API. Per-row insert. Charged per 200 MB. Up to 1 GB/s per table.

### Storage Write API (current)

The modern streaming path. Exactly-once delivery. Cheaper than the legacy streaming insert. Use this for any new streaming workload.

```
BigQueryWriteClient client = BigQueryWriteClient.create();

WriteStream stream = WriteStream.newBuilder()
    .setType(WriteStream.Type.PENDING)
    .build();

WriteStream created = client.createWriteStream(
    CreateWriteStreamRequest.newBuilder()
        .setParent("projects/p/datasets/d/tables/t")
        .setWriteStream(stream)
        .build());
```

Three write modes:

- **Default**: at-least-once delivery, immediate visibility.
- **Pending**: rows buffered until you commit. Exactly-once.
- **Committed**: each row visible immediately, exactly-once via stream offsets.

---

## Views and materialized views

### Views

Saved queries. Resolved at query time. No storage cost. Useful for hiding complex joins or applying row-level security.

```
CREATE VIEW `my-project.analytics.active_users` AS
SELECT id, email, last_login_at
FROM `my-project.analytics.users`
WHERE last_login_at > TIMESTAMP_SUB(CURRENT_TIMESTAMP(), INTERVAL 30 DAY);
```

### Materialized views

Precomputed and incrementally refreshed. Storage cost, but queries are fast and cheap. Good for dashboards.

```
CREATE MATERIALIZED VIEW `my-project.analytics.daily_active_users`
PARTITION BY day
AS
SELECT DATE(login_time) AS day, COUNT(DISTINCT user_id) AS dau
FROM `my-project.analytics.events`
WHERE event_type = 'login'
GROUP BY day;
```

BigQuery automatically substitutes the materialized view when an equivalent query runs against the base table.

## UDFs (User-Defined Functions)

Two languages: SQL and JavaScript.

### SQL UDF

```
CREATE OR REPLACE FUNCTION `my-project.utils.fullName`(first STRING, last STRING)
RETURNS STRING
AS (CONCAT(first, ' ', last));
```

### JavaScript UDF

For logic that's awkward in SQL. Slower than SQL UDFs.

```
CREATE OR REPLACE FUNCTION `my-project.utils.parseUserAgent`(ua STRING)
RETURNS STRUCT<browser STRING, os STRING>
LANGUAGE js
AS r"""
    // ua parsing logic here
    return {browser: 'Chrome', os: 'Mac'};
""";
```

Persistent UDFs live in a dataset and can be called from any project that can read it. Temporary UDFs scope to the current query.

---

## Scheduled queries

Run a query on a schedule, write results to a destination table. Driven by BigQuery Data Transfer Service.

```
-- Configure via UI or `bq mk --transfer_config`
-- Schedule: every day at 02:00 UTC
-- Destination: my-project.analytics.daily_revenue

SELECT DATE(created_at) AS day, SUM(amount) AS revenue
FROM `my-project.payments.charges`
WHERE DATE(created_at) = CURRENT_DATE() - 1
GROUP BY day;
```

Useful for daily rollups, periodic exports, and incremental loads.

---

## Federated queries and external tables

Query data that lives outside BigQuery without loading it.

| Source | Format |
|---|---|
| Cloud Storage | CSV, JSON, Avro, Parquet, ORC |
| Google Drive | Sheets, CSV |
| Cloud SQL | Postgres, MySQL via federated query |
| Bigtable | Direct row reads |
| Cloud Spanner | Read-only |

```
CREATE EXTERNAL TABLE `my-project.raw.events_csv`
OPTIONS (
    format = 'CSV',
    uris = ['gs://my-bucket/events/*.csv'],
    skip_leading_rows = 1
);

SELECT * FROM `my-project.raw.events_csv` LIMIT 100;
```

External tables are slower than native tables (no columnar layout) and cost more (full scan every time, no partition pruning). Use them for staging or rarely-queried data.

---

## Time travel and snapshots

BigQuery keeps every change to a table for 7 days by default. Recover a row that was deleted yesterday:

```
SELECT *
FROM `my-project.analytics.users`
FOR SYSTEM_TIME AS OF TIMESTAMP_SUB(CURRENT_TIMESTAMP(), INTERVAL 1 DAY)
WHERE id = 42;
```

Table snapshots are cheap copy-on-write clones. Use them for backups before risky migrations.

```
CREATE SNAPSHOT TABLE `my-project.backups.users_2025_01_01`
CLONE `my-project.analytics.users`;
```

---

## Caching

Successful queries are cached for 24 hours, per-user. Re-running the same query against unchanged data is free and instant.

Caveats:

- Cache is invalidated by any change to underlying tables.
- Disabled if any query function is non-deterministic (`CURRENT_TIMESTAMP()`, `RAND()`).
- Disabled if you wrote results to a destination table.

Set `useQueryCache=false` to force re-execution.

---

## Java client

```
<dependency>
    <groupId>com.google.cloud</groupId>
    <artifactId>google-cloud-bigquery</artifactId>
    <version>2.x</version>
</dependency>
```

```
BigQuery bq = BigQueryOptions.getDefaultInstance().getService();

QueryJobConfiguration query = QueryJobConfiguration.newBuilder(
        "SELECT id, email FROM `my-project.analytics.users` WHERE status = @status")
    .addNamedParameter("status", QueryParameterValue.string("active"))
    .setUseLegacySql(false)
    .build();

TableResult result = bq.query(query);

for (FieldValueList row : result.iterateAll()) {
    long id = row.get("id").getLongValue();
    String email = row.get("email").getStringValue();
    System.out.println(id + " " + email);
}
```

Use named or positional parameters. String concatenation of user input is a real SQL injection risk in BigQuery just like any other SQL engine.

For high-throughput writes, the Storage Write API (separate library) is the right tool.

---

## Cost control

Top levers, ordered by impact.

1. **Partition every large table.** Day-level partitions on event tables can cut bills by 30x.
2. **Require partition filter on user-facing tables.** Stops accidental full-table scans.
3. **Cluster on high-cardinality filter columns.** Less impactful than partitioning, but free to add.
4. **Avoid `SELECT *`.** Project only the columns you need. Columnar storage rewards narrow selects.
5. **Use materialized views or scheduled rollups for dashboards.** A dashboard scanning the events table every 60 seconds is a slow-motion bill.
6. **Set max bytes billed per query** ( `maximumBytesBilled` in job config) for ad-hoc users. The query fails instead of running away.
7. **Watch the query validator before running.** The UI shows estimated bytes scanned. Don't ignore it.
8. **Move to slot reservations** when on-demand bills become predictable. Flat rate, no surprises.

---

## Query performance tips

- **Filter early.** Push WHERE clauses inside subqueries so less data hits joins.
- **Avoid self-joins on huge tables.** Use window functions instead.
- **Use approximate aggregations.** `APPROX_COUNT_DISTINCT` is around 95% accurate and 100x faster than `COUNT(DISTINCT)`.
- **JOIN on integer keys when possible.** String joins are slower.
- **Look at the query plan.** The execution UI shows stages, slot time, and bytes shuffled. Use it before guessing.
- **Beware skew.** A `GROUP BY user_id` where one user has 90% of rows produces a slow last stage. Salt the key when it matters.

---

## Best practices

- Partition every large table by ingestion time or event time.
- Cluster after partitioning for further pruning.
- Use `require_partition_filter = true` on production tables.
- Project only the columns you need.
- Use parameterized queries from app code.
- Prefer the Storage Write API over the legacy streaming insert.
- Materialized views or scheduled rollups for any query a dashboard hits regularly.
- Keep raw data in dedicated "raw" datasets, transformed data in "curated" datasets.
- Tag jobs with labels ( `team` , `app` , `purpose` ) for cost attribution.
- BigQuery is the warehouse layer. Inserts are slow per-row, reads are fast at scale. Use a different tool for OLTP.

---

## Common gotchas

- **`SELECT *` over a 10 TB table costs around $62.50 per run on on-demand.** Always specify columns.
- **No primary keys, no foreign keys, no UNIQUE constraints.** BigQuery is append-mostly. Dedup with window functions or DML.
- **No traditional indexes.** Performance comes from partitioning, clustering, and projection pushdown.
- **Streaming inserts are buffered.** Newly inserted rows aren't visible to queries for a few seconds.
- **Cross-region queries fail.** Move data to the same location first.
- **`UPDATE` and `DELETE` work but are slow.** BigQuery is meant for append. Heavy update workloads belong elsewhere.
- **`LIMIT` doesn't reduce bytes scanned.** Scanning happens before limiting. Use partitioning for cost. Use LIMIT for sanity.
- **Cost is per query, not per row.** 1 row queried over a 10 GB partition still scans 10 GB. Use WHERE to prune partitions.
- **Concurrent query limits.** On-demand caps at 100 concurrent interactive queries per project. Reservations get higher limits.
- **Scheduled queries can stack.** A scheduled query running over its slot can collide with the next run. Set the schedule with margin.
- **JavaScript UDFs are around 10x slower than SQL UDFs.** Stick to SQL UDFs unless you need JS.

---

## Quick reference

| Concept | Key fact |
|---|---|
| Storage / compute | Decoupled. Pay separately for both. |
| Pricing model | On-demand ($/TB scanned) or slot reservations (flat rate) |
| Partition | Splits a table by a column for cost-effective filtering |
| `require_partition_filter` | Forces queries to include the partition column |
| Cluster | Sorts data within a partition. Up to 4 columns. |
| ARRAY / STRUCT | First-class nested types. `UNNEST` flattens. |
| Materialized view | Precomputed, auto-refreshed table. Worth it for dashboards. |
| Storage Write API | The modern streaming path. Exactly-once. |
| Time travel | Query a table as of any point in the last 7 days |
| Cache | Successful queries cached 24h per-user |
| `APPROX_COUNT_DISTINCT` | Sketch-based, around 100x faster than exact |
| `SELECT *` | Scans every column. Expensive on wide tables. |
| Java client | `com.google.cloud:google-cloud-bigquery`, supports parameterized queries |
| Slot | A virtual CPU + memory unit. Queries consume slots dynamically. |
| Federated query | Query GCS, Drive, Cloud SQL without loading |